● PREMIUM LEARNING GUIDE

The Three Spring Boot Concepts Every Developer Uses But Few Truly Understand

The Request Lifecycle · The Secret Behind @Autowired · What Really Happens When Spring Boot Starts — explained from first principles, one internal mechanism at a time.

```
public class DemoApplication {
  public static void main(String[] args) {
    SpringApplication.run(DemoApplication.class, args);
    // ...this single line does more than most developers realize.
  }
}
```

Muhammad Sufyan · linkedin.com/in/muhammad-sufyan-12b725267

TABLE OF CONTENTS

Three Concepts, One Deeper Understanding

Every Spring Boot developer types these patterns daily. This guide takes them apart, piece by piece, until "it just works" becomes "I know exactly why it works."

PART I · THE COMPLETE REQUEST LIFECYCLE

PART II · THE SECRET BEHIND @AUTOWIRED

PART III · WHAT REALLY HAPPENS WHEN SPRING BOOT STARTS?

CLOSING

◆ **HOW TO READ THIS GUIDE**

Each concept follows the same rhythm: what usually confuses people, the correct mental model, why Spring was designed that way, a real analogy, a code example, and what's happening internally. Read it in order — Part III leans on ideas built in Parts I and II.

PART I

The Complete Spring Boot Request Lifecycle

Every API call you've ever made travels through the same twelve-step journey — from the browser, through layers you rarely think about, to the database and back. This part walks that journey one honest step at a time.

DispatcherServlet | Handler Mapping | @RequestBody / @ResponseBody | Jackson & ObjectMapper

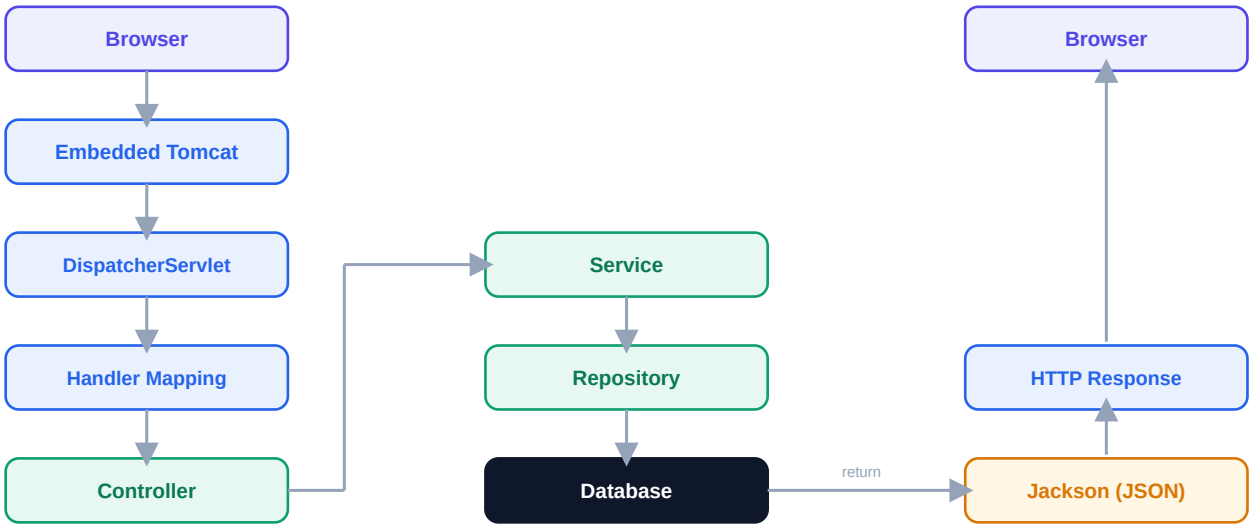
PART I · THE COMPLETE REQUEST LIFECYCLE

1 The Journey of a Single HTTP Request

Type a URL, tap a button, submit a form — and in a few hundred milliseconds, twelve distinct handoffs happen before you see a response. Most developers know the two endpoints (browser and database) and treat everything between them as a black box. This chapter opens the box.

✕ WHAT CONFUSION DID I HAVE?

I used to think "the controller handles the request" — as if nothing meaningful happened before my code ran. In reality, by the time your `@GetMapping` method executes, Spring has already made several decisions on your behalf.



Solid path = the request going down · Return path = the response coming back up

◆ WHY DOES SPRING WORK THIS WAY?

Each layer has exactly one job — Tomcat handles raw sockets, DispatcherServlet routes, the Controller talks HTTP, the Service holds business rules, the Repository talks to data. Splitting responsibilities this way means each piece can change independently without breaking the others.

PART I · THE COMPLETE REQUEST LIFECYCLE

STEP 1

The Browser Sends an HTTP Request

Before any Spring code runs, the browser packages your action — a page load, a form submit, a button click behind a fetch call — into a plain text HTTP request and sends it over TCP to your server's port (usually 8080 locally).

✓ WHAT IS THE CORRECT UNDERSTANDING?

An HTTP request is just structured text: a method (GET, POST...), a path, headers, and optionally a body. Spring never sees your browser directly — it only ever sees this text, parsed into Java objects.

GET

```
GET /api/journal/42 HTTP/1.1
Host: localhost:8080
Accept: application/json
```

Used to **retrieve** data. No body. Parameters travel in the URL or query string. Safe to repeat — it shouldn't change anything on the server.

POST

```
POST /api/journal HTTP/1.1
Host: localhost:8080
Content-Type: application/json

{"title":"Day One","mood":"calm"}
```

Used to **create or change** data. Carries a body — usually JSON — that Spring will convert into a Java object for you.

◆ REAL-WORLD ANALOGY

A GET request is like handing a librarian a call number and asking to see the book — you're not changing the shelf. A POST request is like handing over a new book to be added to the collection — you're bringing something new for the system to store.

◆ DID YOU KNOW?

HTTP itself is stateless and human-readable — you can literally open a terminal, run `telnet localhost 8080`, type a GET request by hand, and get a real response back. Spring Boot's abstractions sit entirely on top of this plain text protocol.

⚙ INTERNAL PROCESS

The browser resolves the host, opens a TCP connection, writes the request line + headers + body as raw bytes, and waits. Nothing Spring-specific has happened yet — this is pure networking, identical whether the server is Java, Node, or Python.

STEP 2

Embedded Tomcat — The Gatekeeper

The raw bytes from the browser have to land somewhere. In a Spring Boot app, that "somewhere" is an embedded Tomcat server — a full servlet container packed inside your JAR, started automatically when your app boots.

✕ WHAT CONFUSION DID I HAVE?

I assumed "Spring Boot" and "the web server" were the same thing. They're not. Spring Boot is a framework that configures and wires your application; Tomcat is a separate, mature piece of software that Spring Boot simply starts and hands requests to.

✓ WHAT IS THE CORRECT UNDERSTANDING?

Tomcat's job stops at the network layer: accept the TCP connection, parse the raw bytes into an `HttpServletRequest` object, and hand that object to whichever servlet is registered to handle it. In a Spring Boot app, there is exactly one such servlet — the DispatcherServlet.

What Tomcat Does	What Tomcat Does NOT Do
Accepts TCP connections on a port	Know anything about your @Controller classes
Parses raw bytes into HttpServletRequest	Understand JSON, DTOs, or your business logic
Manages threads for concurrent requests	Decide which Java method should run
Sends the final response bytes back	Talk to your database

◆ WHY DOES SPRING WORK THIS WAY?

Before Spring Boot, you had to install Tomcat separately, and deploy a `.war` file into it. Embedding Tomcat inside a runnable JAR removed that entire setup step — "it just runs" became literally true, with `java -jar app.jar` starting both your code and its own web server together.

⚠ WHAT HAPPENS IF THIS COMPONENT DIDN'T EXIST?

Without an embedded servlet container, your Spring Boot application would just be a plain Java program — no port would ever be opened, and no HTTP request could ever reach your code. You'd need to manually deploy your app into a separately installed server.

STEP 3

DispatcherServlet — The Front Controller

Tomcat hands the parsed request to exactly one servlet: Spring's `DispatcherServlet`. This is arguably the single most important class in all of Spring MVC — every request for every controller in your app funnels through this one object.

✓ WHAT IS THE CORRECT UNDERSTANDING?

DispatcherServlet implements the **Front Controller** pattern — instead of Tomcat needing to know about dozens of your controller classes, it only needs to know about one servlet. That servlet then takes responsibility for routing the request onward to the correct place.

```
// registered automatically by Spring Boot auto-configuration —
// you never write this class yourself
public class DispatcherServlet extends FrameworkServlet {
    protected void doService(HttpServletRequest request,
                             HttpServletResponse response) {
        // 1. find a HandlerMapping match
        // 2. invoke the matched Controller method
        // 3. hand the return value to a view resolver or JSON writer
    }
}
```

What DispatcherServlet actually coordinates, in order:

1. **Handler Mapping** — asks "which controller method matches this URL and HTTP method?"
2. **Handler Adapter** — actually invokes that method, resolving its parameters
3. **Exception Resolver** — catches and converts any thrown exception into an HTTP error response
4. **Response Writer** — serializes the returned object (via Jackson) back into the HTTP response

◆ REAL-WORLD ANALOGY

Think of a large hospital's front reception desk. Patients don't wander the building looking for the right doctor — they check in at one desk, which reads their symptoms and routes them to the correct department. DispatcherServlet is that reception desk for every request in your app.

⚠ WHAT HAPPENS IF THIS COMPONENT DIDN'T EXIST?

Without a front controller, every incoming URL would need its own dedicated servlet registered directly with Tomcat — no shared request-handling pipeline, no centralized exception handling, and no single place to plug in filters, interceptors, or JSON conversion.

PART I · THE COMPLETE REQUEST LIFECYCLE

STEP 4

Handler Mapping — Finding the Right Method

DispatcherServlet doesn't know your code. It asks a component called **HandlerMapping** a simple question: "for this URL and this HTTP method, which exact Java method should run?"

✕ WHAT CONFUSION DID I HAVE?

I imagined Spring "searching" your whole codebase on every request, which sounded slow. In reality, this lookup is closer to a dictionary lookup than a search — the mapping is built once, at startup, and reused for every request.

✓ WHAT IS THE CORRECT UNDERSTANDING?

At startup, Spring scans every `@Controller` / `@RestController` class, reads each method's `@GetMapping`, `@PostMapping`, etc., and builds an in-memory table: URL pattern + HTTP method → the exact method to call. At request time, it's just a lookup in that table.

URL Pattern	HTTP Method	Maps To
`/api/journal`	GET	`JournalController.getAll()`
`/api/journal/{id}`	GET	`JournalController.getById()`
`/api/journal`	POST	`JournalController.create()`

```
@RestController
@RequestMapping("/api/journal")
public class JournalController {

    @GetMapping("/{id}")
    public JournalEntry getById(@PathVariable String id) { ... }
}
```

◆ WHY DOES SPRING WORK THIS WAY?

Building the routing table once at startup — instead of re-scanning annotations on every request — is what makes routing fast. It also means a broken or ambiguous mapping (two methods claiming the same URL + method) fails loudly at startup, not silently in production.

🎓 INTERVIEW TIP

If asked "what happens if two controller methods map to the same URL and HTTP method?" — the answer is: Spring Boot fails to start, throwing an `IllegalStateException` about ambiguous mappings, rather than silently picking one at runtime.

STEP 5

The Controller Layer & @RequestBody

Once Handler Mapping identifies the target method, DispatcherServlet's Handler Adapter actually invokes it — but first it must resolve every parameter that method asks for, including one very common case: a JSON request body.

✓ WHAT IS THE CORRECT UNDERSTANDING?

A Controller's only responsibility is to speak HTTP: read what came in (path variables, query params, JSON body), call the Service layer to do the actual work, and shape what goes out. It should contain almost no business logic itself.

```
@RestController
@RequestMapping("/api/journal")
public class JournalController {

    private final JournalService service;

    @PostMapping
    public JournalEntry create(@RequestBody JournalEntry entry) {
        return service.save(entry);
    }
}
```

◆ WHY DOES SPRING WORK THIS WAY?

`@RequestBody` tells Spring: "take the raw JSON text sitting in the HTTP request body, and convert it into a `JournalEntry` Java object before calling my method." Without it, the parameter would just be treated as a query parameter, and the incoming JSON would be ignored entirely.

INCOMING JSON

```
{
  "title": "Day One",
  "mood": "calm"
}
```

RESULTING JAVA OBJECT

```
JournalEntry {
  title = "Day One"
  mood  = "calm"
}
```

This JSON → Java conversion is performed by Jackson, which Chapter 9 covers in detail.

⚠ COMMON BEGINNER MISTAKE

Forgetting `@RequestBody` on a POST parameter is one of the most common early bugs — the method compiles fine, but the parameter silently arrives as `null` because Spring never looked at the request body for it.

STEP 6

The Service Layer — Where Business Logic Lives

The Controller hands off to a Service. This is the layer beginners most often skip — "why not just call the repository directly from the controller?" — but it exists for a very deliberate reason.

✕ WHAT CONFUSION DID I HAVE?

Early on, my "service" classes were just thin pass-throughs to the repository, and I genuinely couldn't explain what problem the extra layer solved. It felt like ceremony for its own sake.

✓ WHAT IS THE CORRECT UNDERSTANDING?

The Service layer owns **business rules** — validation, calculations, coordinating multiple repositories, enforcing what's allowed. The Controller shouldn't know these rules; it should just trust the Service to apply them.

```
@Service
public class JournalService {

    private final JournalEntryRepository repository;

    public JournalEntry save(JournalEntry entry) {
        if (entry.getTitle() == null || entry.getTitle().isBlank()) {
            throw new IllegalArgumentException("Title is required");
        }
        entry.setCreatedAt(Instant.now());
        return repository.save(entry);
    }
}
```

◆ WHY DOES SPRING WORK THIS WAY?

Separating "what HTTP looks like" (Controller) from "what the business allows" (Service) from "how data is stored" (Repository) means you can test business rules without spinning up a web server, and swap the database without touching a single business rule.

◆ REAL-WORLD ANALOGY

Think of a bank. The teller (Controller) takes your request at the counter. The underwriting department (Service) decides if a loan is actually approvable, applying the bank's real rules. The vault records system (Repository) simply stores the final numbers — it has no opinion about whether the loan should have been approved.

◆ DID YOU KNOW?

A Controller is technically allowed to call a Repository directly — Spring won't stop you. The layering is a convention, not a compiler-enforced rule. That's exactly why skipping it doesn't "error out"; it just quietly makes the codebase harder to maintain.

PART I · THE COMPLETE REQUEST LIFECYCLE

STEP 7

The Repository Layer & The Database Round-Trip

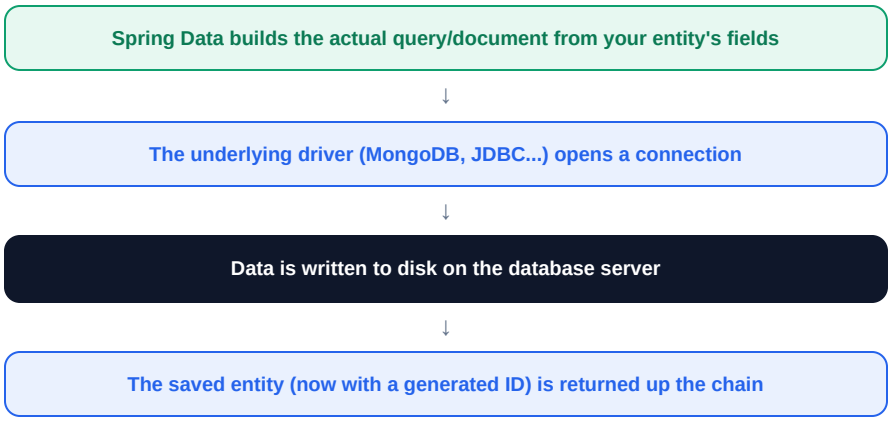
The Service hands the object to a Repository — the only layer that's allowed to know how data is actually stored.

```
public interface JournalEntryRepository extends
        MongoRepository<JournalEntry, String> { }
```

✓ WHAT IS THE CORRECT UNDERSTANDING?

You never implement this interface. At startup, Spring Data generates a real class behind the scenes that implements `save()`, `findById()`, `findAll()`, and `deleteById()` — each one correctly typed to work with `JournalEntry` and its `String` id, because of the Generic type parameters you supplied.

What happens on `repository.save(entry)`, step by step:



◆ HOW DOES THIS RELATE TO SPRING BOOT?

This is the exact line that this guide's companion book on Generics unpacks in depth: `MongoRepository<T, ID>` is a generic interface, and every method it hands you is type-safe because you told it, once, what `T` and `ID` mean for your entity.

⚠ WHAT HAPPENS IF THIS COMPONENT DIDN'T EXIST?

Without Spring Data's auto-generated implementations, you'd hand-write boilerplate CRUD methods for every entity in your app — opening connections, mapping result rows to objects, and closing resources manually for each one.

PART I · THE COMPLETE REQUEST LIFECYCLE

STEP 8

Jackson & ObjectMapper — JSON, Decoded

Every time JSON turns into a Java object (or back), a library called **Jackson** is doing the work — quietly, automatically, and by default without you writing a single line of conversion code.

✕ WHAT CONFUSION DID I HAVE?

I assumed Spring itself parsed JSON. It doesn't — Spring Boot simply includes Jackson on the classpath by default and wires it in as the tool responsible for HTTP message conversion.

✓ WHAT IS THE CORRECT UNDERSTANDING?

Jackson's core class is `ObjectMapper`. It has two directions: **deserialization** (JSON text → Java object) for incoming `@RequestBody` data, and **serialization** (Java object → JSON text) for outgoing responses. Spring calls this for you behind the scenes on every request and response.

Direction	Trigger	Jackson Operation
Incoming JSON → Java	`@RequestBody`	Deserialization
Java → Outgoing JSON	`@ResponseBody` / `@RestController`	Serialization

```
// this is roughly what Spring does internally for you
ObjectMapper mapper = new ObjectMapper();

// deserialization: JSON text -> Java object
JournalEntry entry = mapper.readValue(jsonString, JournalEntry.class);

// serialization: Java object -> JSON text
String json = mapper.writeValueAsString(entry);
```

◆ WHY DOES SPRING WORK THIS WAY?

Jackson uses reflection to match JSON field names to your class's getters/setters (or fields) by name. This is why a field named `title` in your class must match a JSON key named `"title"` — there's no magic mapping beyond that name match (or an explicit `@JsonProperty` override).

🎓 INTERVIEW TIP

If asked "how does Spring convert JSON to Java objects automatically?" — mention **HttpMessageConverter**: Jackson's converter is registered as one of these, and DispatcherServlet consults it whenever a method has an `@RequestBody` parameter or returns a response body.

STEP 9

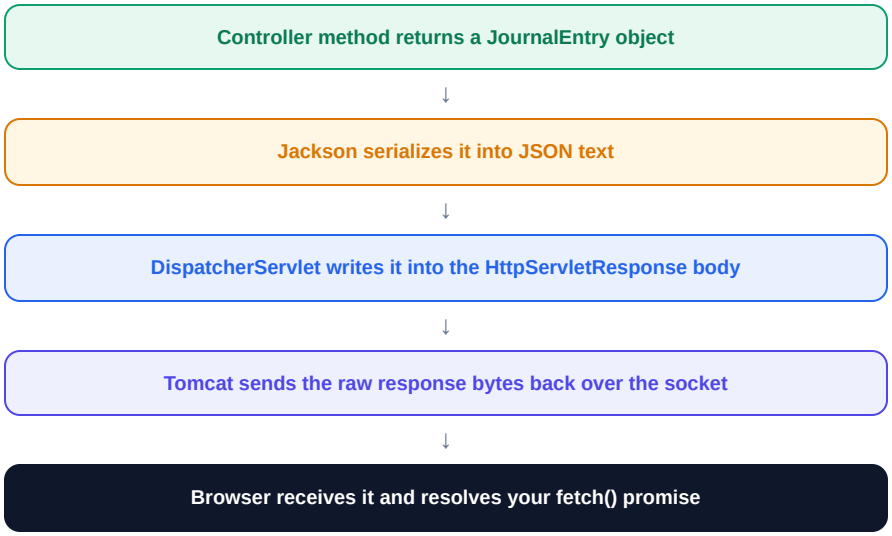
@ResponseBody & The Journey Back to the Browser

Your Controller method returns a plain Java object. That object now has to travel all the way back down through Tomcat and out onto the wire as JSON text — this is the mirror image of Step 5.

```
@RestController   // = @Controller + @ResponseBody on every method
public class JournalController {

    @GetMapping("/{id}")
    public JournalEntry getById(@PathVariable String id) {
        return service.findById(id);   // just a Java object — not JSON yet
    }
}
```

✓ WHAT IS THE CORRECT UNDERSTANDING?

`@ResponseBody` tells Spring: "don't treat this return value as the name of an HTML view to render — treat it as data, and write it directly into the HTTP response body." `@RestController` applies this to every method in the class automatically, which is why REST APIs almost never write `@ResponseBody` explicitly.



◆ REAL-WORLD ANALOGY

It's the hospital again: the doctor's diagnosis (your Java object) means nothing to the patient until the front desk translates it into a discharge form (JSON) the patient can actually carry home and read.

STEP 10

Putting It All Together — A Full Round Trip

Here's the entire journey for one concrete POST request, with every layer's job stated in one line.

#	Layer	What Happens Here
1	Browser	Sends `POST /api/journal` with a JSON body
2	Embedded Tomcat	Accepts the connection, builds an HttpServletRequest
3	DispatcherServlet	Receives the request, begins the dispatch process
4	Handler Mapping	Matches POST + `/api/journal` → `create()`
5	Jackson	Deserializes the JSON body into a `JournalEntry`
6	Controller	`create()` calls `service.save(entry)`
7	Service	Validates the title, sets a timestamp
8	Repository	Persists the entry to MongoDB, returns it with an ID
9	Jackson	Serializes the saved entry back into JSON
10	Browser	Receives the JSON response with the new ID

⚠ WHAT IF A COMPONENT WERE MISSING?

Remove any single layer from this chain and something breaks specifically: no Tomcat → no port ever opens. No DispatcherServlet → Tomcat has nothing to hand requests to. No Handler Mapping → every request 404s. No Jackson → your API can only speak plain text, not JSON. Each layer is load-bearing.

◆ DID YOU KNOW?

You can watch nearly this entire pipeline in a debugger. Set a breakpoint inside `DispatcherServlet.doDispatch()` (yes, it's real, open-source code you can step into) and you'll see Handler Mapping, the Handler Adapter, and your own Controller method get invoked in exactly this order.

🎓 INTERVIEW TIP

A strong answer to "walk me through what happens when a Spring Boot API receives a request" is exactly this ten-row table — most candidates can only describe steps 6–8. Being able to describe steps 1–5 and 9–10 as well is what separates "used Spring" from "understands Spring."

CHAPTER CLOSE

Common Beginner Mistakes & Chapter Summary

⚠ SIX MISTAKES WORTH WATCHING FOR

1. Believing the Controller is where the request "starts." Four steps (Tomcat, DispatcherServlet, Handler Mapping, parameter resolution) happen before your method body runs.

2. Forgetting @RequestBody on a POST parameter. The method compiles, but the parameter arrives as null — Spring never read the request body for it.

3. Putting business rules directly in the Controller. Skipping the Service layer works today and makes testing and reuse harder tomorrow.

4. Assuming Spring parses JSON itself. It's Jackson — a separate library Spring Boot wires in for you by default.

5. Thinking GET requests can safely carry a body. While technically possible, GET is meant to be side-effect-free and many tools/proxies strip or ignore GET bodies.

6. Assuming a repository method is hand-written. Interfaces like `MongoRepository<T, ID>` are implemented for you at startup — you never provide a class body.

✎ CHAPTER SUMMARY

- Every request passes through Tomcat → DispatcherServlet → Handler Mapping → Controller → Service → Repository → Database, then back.
- Tomcat only understands bytes and connections — it knows nothing about your Spring code.
- DispatcherServlet is the single Front Controller that every request funnels through.
- Handler Mapping is a routing table built once at startup, not searched per-request.
- Jackson's ObjectMapper handles both directions of JSON conversion — deserialization in, serialization out.
- @RequestBody and @ResponseBody are the two annotations that connect your Java objects to that JSON conversion.
- Each layer exists to isolate one concern — removing any one of them breaks a specific, predictable part of the chain.

◆ LOOKING AHEAD

Every layer in this chapter — Controller, Service, Repository — was created by Spring and handed to you fully wired, with its dependencies already filled in. Part II opens up exactly how that happens: the secret behind `@Autowired`.

PART II

The Secret Behind @Autowired

Every Controller, Service, and Repository from Part I arrived on the scene already built and already connected to each other. Nobody called "new" on any of them. This part explains who did — and how.

IoC Container | ApplicationContext | Bean Lifecycle | Dependency Injection

2 The Problem With "new"

Look back at the `JournalService` from Part I. It needed a `JournalEntryRepository` to do its job. Nowhere in that class did anyone write `new JournalEntryRepository()`. So where did it come from?

✕ WHAT CONFUSION DID I HAVE?

I could write `@Autowired` and watch it work, but I genuinely didn't know what would break if I tried to build these objects myself with `new` instead. It felt like a rule to follow, not a mechanism to understand.

Here's what building everything manually would actually look like:

```
public class JournalService {
    private final JournalEntryRepository repository = new JournalEntryRepositoryImpl(new MongoClient(...));
}

public class JournalController {
    private final JournalService service = new JournalService(new JournalEntryRepositoryImpl(new
MongoClient(...)));
}
```

◆ WHY DOES SPRING WORK THIS WAY?

Every class above has to know how to construct all of its dependencies, and their dependencies, all the way down. Every constructor signature change ripples through every caller. Testing becomes painful — you can't easily swap in a fake repository. Spring's answer: let one central system build every object once, and simply **hand** each class what it needs.

Manual "new"	Spring-Managed
Each class builds its own dependencies	One container builds every object, once
Constructor changes ripple everywhere	Objects just declare what they need
Hard to swap implementations for tests	Swap easily by registering a different bean
No shared lifecycle or configuration	Centralized creation, configuration, and teardown

◆ DID YOU KNOW?

This general idea — code declaring what it needs instead of constructing it — has a name older than Spring itself: **Inversion of Control (IoC)**. Spring didn't invent the concept; it built one of the most popular implementations of it for Java.

PART II · THE SECRET BEHIND @AUTOWIRED

STEP 1

The IoC Container — Who's Really in Charge?

The "one central system" from the previous page has a name: the **IoC Container**. It is, quite literally, the thing that runs your entire Spring Boot application from the inside.

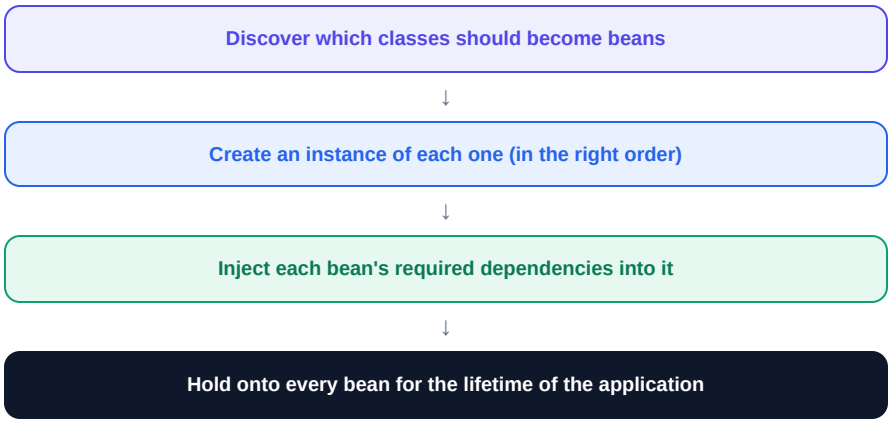
✓ WHAT IS THE CORRECT UNDERSTANDING?

The IoC Container is responsible for three jobs: **creating** objects, **configuring** them, and **wiring** them together — connecting each object to the other objects it depends on. Every object it manages is called a **bean**.

◆ REAL-WORLD ANALOGY

Without the container, you're a chef who has to grow their own vegetables, raise their own livestock, and forge their own knives before cooking a single dish. With the container, you walk into a fully stocked kitchen — every ingredient and tool is already there, ready, the moment you need it.

The container's core cycle, every time your app starts:



◆ WHY DOES SPRING WORK THIS WAY?

Centralizing object creation means your business classes stop caring about "how do I build my dependencies" and only care about "what do I need to do my job." That single shift is what makes Spring code so much shorter than manually-wired Java.

🎓 INTERVIEW TIP

"What is IoC?" is one of the most common Spring interview questions. The precise answer: Inversion of Control means the framework controls object creation and wiring, instead of your code controlling it — the control has been "inverted" away from you.

STEP 2

ApplicationContext — The Container in Action

"IoC Container" is the concept. `ApplicationContext` is the actual Java interface Spring gives you for it — the real, running object that holds every bean in your application.

✕ WHAT CONFUSION DID I HAVE?

I treated "IoC Container," "ApplicationContext," and "Spring context" as three unrelated buzzwords I'd heard in different tutorials. They're the same thing, described at different levels of precision.

✓ WHAT IS THE CORRECT UNDERSTANDING?

`ApplicationContext` is an interface; Spring Boot creates a concrete implementation of it (typically `AnnotationConfigServletWebServerApplicationContext`) the moment `SpringApplication.run()` executes. From that point on, it is the live registry of every bean in your app.

```
// you rarely need to touch ApplicationContext directly, but you can:
ApplicationContext context = SpringApplication.run(DemoApplication.class, args);

JournalService service = context.getBean(JournalService.class);
// this is the exact same instance @Autowired would have given you
```

◆ REAL-WORLD ANALOGY

If the IoC Container is the idea of "a company has an org chart," ApplicationContext is the actual, physical HR database that lists every employee, their role, and who reports to whom — the live, queryable version of the idea.

◆ DID YOU KNOW?

ApplicationContext extends a simpler interface called `BeanFactory`. BeanFactory only knows how to create and fetch beans; ApplicationContext adds event handling, internationalization, and — critically for web apps — integration with the servlet environment.

⚠ COMMON BEGINNER MISTAKE

Manually calling `context.getBean(...)` everywhere, instead of letting `@Autowired` inject dependencies for you, is a red flag called the **Service Locator anti-pattern** — it hides a class's real dependencies instead of declaring them up front.

STEP 3

Stereotype Annotations — Marking the Beans

ApplicationContext needs to know which of your classes it should manage. You tell it using **stereotype annotations** — small markers placed directly above a class.

✓ WHAT IS THE CORRECT UNDERSTANDING?

Any class annotated with `@Component` (or one of its specialized variants) becomes eligible to be turned into a bean during startup scanning. Without one of these annotations, Spring simply doesn't know the class exists.

Annotation	Meaning	Typical Use
`@Component`	Generic Spring-managed bean	Any general-purpose class
`@Controller`	Handles web requests, returns views	Traditional MVC web controllers
`@RestController`	@Controller + @ResponseBody on every method	REST API controllers
`@Service`	Holds business logic	The Service layer from Part I
`@Repository`	Talks to the data store	The Repository layer from Part I

◆ WHY DOES SPRING WORK THIS WAY?

All five annotations do the exact same core thing technically — register the class as a bean — because `@Controller`, `@Service`, and `@Repository` are themselves internally annotated with `@Component`. The different names exist purely for **readability and intent**, and a couple of them (like `@Repository`) add small extra behaviors, such as translating database exceptions into a consistent Spring exception type.

```
// simplified — this is really what @Service looks like internally
@Target(ElementType.TYPE)
@Retention(RetentionPolicy.RUNTIME)
@Component   // <- @Service is built on top of @Component
public @interface Service { }
```

🎓 INTERVIEW TIP

"What's the difference between @Component and @Service?" — functionally, almost nothing. The honest answer is that they're semantically distinct (readable intent) rather than technically distinct, aside from @Repository's exception translation.

PART II · THE SECRET BEHIND @AUTOWIRED

STEP 4

@ComponentScan — How Spring Finds Your Classes

Marking a class with `@Service` is only half the story. Something still has to physically look through your compiled code and find every class carrying one of these annotations. That something is `@ComponentScan`.

✕ WHAT CONFUSION DID I HAVE?

I never saw `@ComponentScan` written anywhere in my projects, so I assumed it wasn't happening. It was — it's silently included inside `@SpringBootApplication`, the one annotation every Spring Boot main class has.

```
@SpringBootApplication   // this single annotation is actually three:
// @Configuration + @EnableAutoConfiguration + @ComponentScan
public class DemoApplication {
    public static void main(String[] args) {
        SpringApplication.run(DemoApplication.class, args);
    }
}
```

✓ WHAT IS THE CORRECT UNDERSTANDING?

By default, `@ComponentScan` scans the **package containing your main class, and every sub-package beneath it** — using reflection to open each `.class` file and check for stereotype annotations. Anything found is registered as a bean definition.

⚠ COMMON BEGINNER MISTAKE

Placing a `@Service` or `@Repository` class in a package **outside** the main class's package tree is a classic bug source — Spring simply never sees it, and any attempt to `@Autowired` it fails with `NoSuchBeanDefinitionException`.

◆ WHY DOES SPRING WORK THIS WAY?

Scanning by package convention avoids the need to manually list every single class you want managed. As your app grows to hundreds of classes, you'd never want to maintain that list by hand — you just follow the package structure convention, and scanning does the rest.

STEP 5

@Configuration & @Bean — Manual Bean Creation

Not everything can be marked with `@Component` — you can't add an annotation to a class from an external library, or a class that needs custom construction logic. For these cases, Spring gives you a manual escape hatch.

✓ WHAT IS THE CORRECT UNDERSTANDING?

A class marked `@Configuration` is itself treated as a special kind of component. Any method inside it marked `@Bean` tells Spring: "call this method once, and register whatever it returns as a bean."

```
@Configuration
public class AppConfig {

    @Bean
    public ObjectMapper objectMapper() {
        ObjectMapper mapper = new ObjectMapper();
        mapper.registerModule(new JavaTimeModule());
        return mapper;   // Spring now manages this instance as a bean
    }
}
```

@Component (on the class)	@Bean (on a method)
You own and can annotate the class	Class comes from a third-party library, or needs custom setup
Discovered automatically via scanning	Explicitly declared, method by method
Instance is created by calling the no-arg constructor	Instance is whatever the method body returns

◆ REAL-WORLD ANALOGY

@Component is like a job applicant who fills out the standard form and gets auto-processed by HR. @Bean is like a specialist hire brought in through a manual, hand-written offer letter — same outcome (they join the company/context), different path to get there.

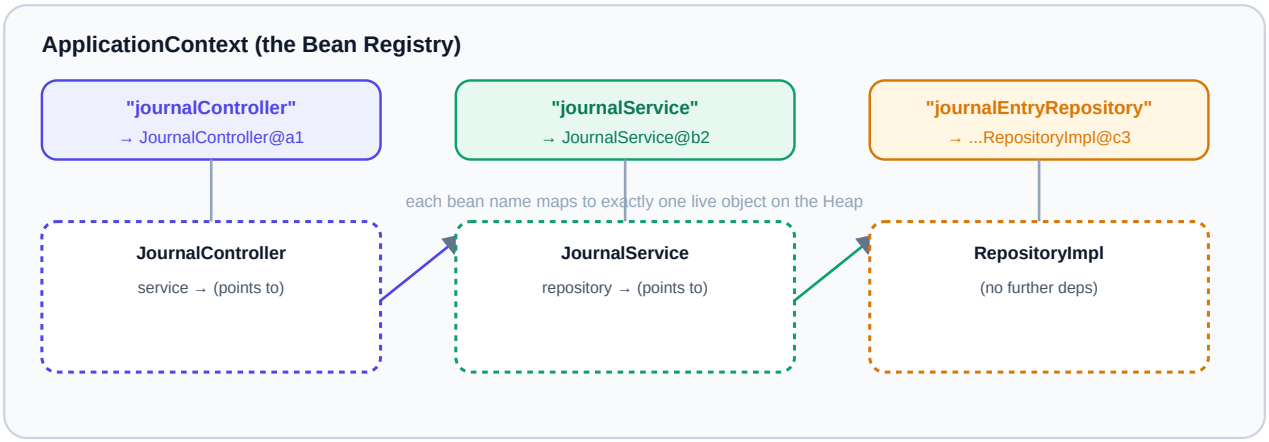
◆ DID YOU KNOW?

Spring Boot's own auto-configuration (covered in Part III) is built almost entirely out of `@Configuration` classes full of `@Bean` methods — that embedded Tomcat instance from Part I is itself registered as a bean this exact way.

STEP 6

Where Do Beans Actually Live?

Every bean is a normal Java object sitting on the Heap — nothing magical about its memory location. What's different is that ApplicationContext keeps a reference to each one in an internal registry, so it never gets garbage collected while your app is running.



✓ WHAT IS THE CORRECT UNDERSTANDING?

Beans reference each other exactly like normal Java objects reference each other (Generics book, Chapter 2) — via reference variables holding a Heap address. The only difference is **who set that reference**: normally you write the assignment yourself; here, the container does it for you.

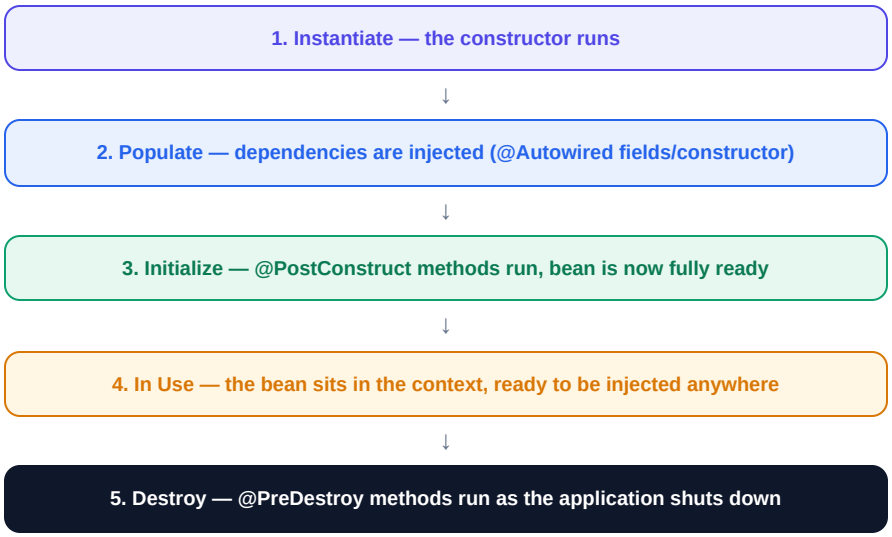
◆ WHY DOES SPRING WORK THIS WAY?

Naming each bean (usually derived from the class name) lets the container disambiguate when multiple beans of a compatible type exist, and lets you fetch a specific one by name when needed.

STEP 7

The Bean Lifecycle

A bean isn't just instantiated and forgotten — it moves through a defined sequence of stages, and Spring gives you hooks into several of them.




```java
@Service
public class JournalService {

    @PostConstruct
    public void init() {
        System.out.println("JournalService is fully wired and ready.");
    }

    @PreDestroy
    public void cleanup() {
        System.out.println("Application shutting down — releasing resources.");
    }
}
```

◆ WHY DOES SPRING WORK THIS WAY?

Splitting "construct" from "initialize" matters because dependency injection has to finish first — you can't safely use an injected field inside a constructor if Spring hasn't set it yet. `@PostConstruct` guarantees all dependencies are already in place before your setup logic runs.

⚠ COMMON BEGINNER MISTAKE

Trying to use an `@Autowired` field inside the constructor body of the same class. Field injection happens **after** construction — inside the constructor, that field is still `null`.

PART II · THE SECRET BEHIND @AUTOWIRED

STEP 8

Singleton Scope — One Bean, Many Injections

If ten different classes all declare a dependency on `JournalService`, how many `JournalService` objects does Spring create?

✕ WHAT CONFUSION DID I HAVE?

I assumed a new instance was created every time it was injected somewhere — the same way `new Student()` creates a fresh object every single time it runs.

✓ WHAT IS THE CORRECT UNDERSTANDING?

By default, every Spring bean is a **singleton** — exactly one instance is created for the entire application, and every class that needs it gets a reference to that same shared object. Ten classes injecting `JournalService` get ten references pointing at one object on the Heap.

```
// all three of these fields point to the exact same object in memory
@RestController
public class JournalController {
    private final JournalService service; // same instance
}

@Component
public class ReminderScheduler {
    private final JournalService service; // same instance
}
```

◆ WHY DOES SPRING WORK THIS WAY?

Most beans are stateless (Services, Repositories, Controllers) — they don't hold per-request data as instance fields, so sharing one instance is both safe and far cheaper than constructing a fresh object on every injection.

⚠ COMMON BEGINNER MISTAKE

Storing per-request or per-user state as an instance field on a default (singleton-scoped) `@Service` or `@Controller` — since it's shared across every request and every user, this creates subtle, hard-to-reproduce data leaks between unrelated requests.

◆ DID YOU KNOW?

Singleton isn't the only scope. `@Scope("prototype")` creates a brand new instance on every injection, and web-specific scopes like `request` and `session` tie a bean's lifetime to a single HTTP request or user session.

STEP 9

The Secret Behind @Autowired, Finally

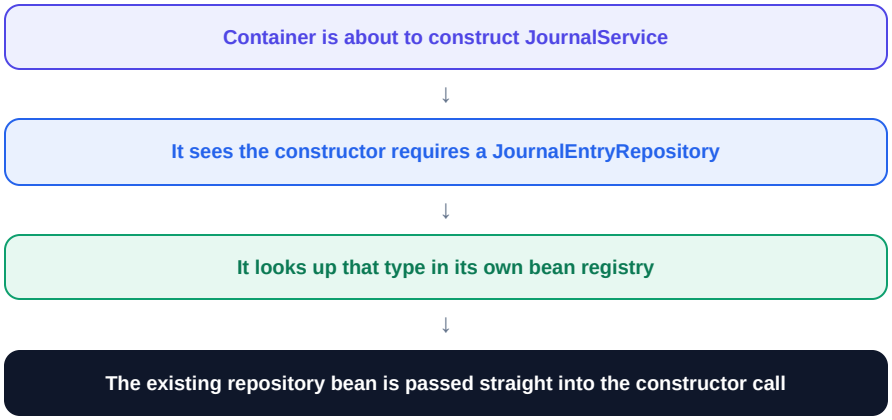
Every piece is now in place — beans, ApplicationContext, singleton scope. Time to answer the question that opened this Part: what does `@Autowired` actually do?

```
@Service
public class JournalService {

    private final JournalEntryRepository repository;

    @Autowired  // optional on a single constructor since Spring 4.3+
    public JournalService(JournalEntryRepository repository) {
        this.repository = repository;
    }
}
```

✓ WHAT IS THE CORRECT UNDERSTANDING?

`@Autowired` tells Spring: "before you finish constructing this bean, look at what type this parameter (or field) needs, find a matching bean already sitting in ApplicationContext, and pass it in." It's a request to the container, resolved entirely by **type** (and by name, if there's more than one match).

What actually happens, step by step, when JournalService is created:



◆ WHY DOES SPRING WORK THIS WAY?

Under the hood, this uses **reflection** — the same mechanism Jackson used in Part I to match JSON fields. Spring inspects constructor parameter types (or field types) at startup and resolves each one against its bean registry, entirely through Java's reflection API.

🎓 INTERVIEW TIP

Constructor injection (shown above) is the officially recommended style over field injection (`@Autowired` directly on a field) — it makes dependencies explicit, allows fields to be `final`, and makes classes easier to unit test without starting a full Spring context.

STEP 10

"new" vs Spring-Managed Objects

Both approaches ultimately do the same fundamental thing — create an object on the Heap and store its address in a reference variable (Generics book, Chapter 2). What differs is **who initiates it, when, and how many times**.

	`new JournalService()`	@Autowired JournalService
Who creates it	Your code, explicitly	The IoC Container, automatically
When	Whenever that line executes	Once, during application startup
How many instances	One per "new" call — as many as you write	One, shared (default singleton scope)
Dependency wiring	You manually pass in every dependency	The container resolves and injects them for you
Lifecycle hooks	None built-in	@PostConstruct / @PreDestroy supported
Testability	Hard to substitute fakes without changing code	Easy to swap beans for test doubles

◆ REAL-WORLD ANALOGY

"new" is renting your own car every time you need one — you handle the paperwork, insurance, and upkeep yourself, every single time. A Spring bean is like a company car pool — one fleet, professionally maintained, and simply handed to whoever needs it, when they need it.

◆ DID YOU KNOW?

You can still use `new` inside Spring code — for simple data objects like a `JournalEntry` instance, this is completely normal and expected. It's specifically the classes that need **injected dependencies**

⚠ COMMON BEGINNER MISTAKE

Manually calling `new JournalService()` inside a Controller "just to try something quickly." The resulting object is a completely separate, unmanaged instance — its own dependencies won't be auto-injected either, since it never passed through the container.

CHAPTER CLOSE

Common Beginner Mistakes & Chapter Summary

⚠ SIX MISTAKES WORTH WATCHING FOR

1. Placing beans outside the scanned package tree. If a class sits outside the main class's package (or its sub-packages), @ComponentScan never finds it.

2. Using an @Autowired field inside its own constructor. Field injection completes after construction — the field is still null at that point.

3. Storing per-request state on a singleton bean. Default-scoped beans are shared across every user and request; instance fields become a shared, leaking data source.

4. Manually "new"-ing a class that should be a bean. The resulting object never gets its own dependencies injected, since it bypassed the container entirely.

5. Assuming @Component and @Service behave differently. They're functionally almost identical — the distinction is readability, not mechanics.

6. Overusing context.getBean() instead of injection. This "Service Locator" pattern hides a class's real dependencies instead of declaring them.

✎ CHAPTER SUMMARY

- The IoC Container creates, configures, and wires every bean in your application — you stop manually building dependency chains.
- ApplicationContext is the concrete, running implementation of the IoC Container concept.
- @Component (and its specialized forms @Service, @Repository, @Controller) mark a class as eligible to become a bean.
- @ComponentScan, bundled inside @SpringBootApplication, discovers those marked classes automatically.
- @Configuration + @Bean is the manual alternative for classes you can't annotate directly.
- Beans are singleton by default — one shared instance, injected everywhere it's needed.
- @Autowired resolves dependencies by type through reflection, and injects the existing bean instance — it never creates a fresh one per injection.

◆ LOOKING AHEAD

Everything in this chapter happens *during* application startup — but what actually triggers it? Part III opens up the one line every Spring Boot app starts with: `SpringApplication.run()`.

PART III

What Really Happens When Spring Boot Starts?

Parts I and II described a running application. This part rewinds to the very first line — `SpringApplication.run()` — and walks through everything that happens in the seconds before your app says it's ready.

SpringApplication.run() | Auto Configuration | Startup Sequence

3 The One Line: SpringApplication.run()

Every Spring Boot application begins with a main method that's almost suspiciously short:

```
@SpringBootApplication
public class DemoApplication {
    public static void main(String[] args) {
        SpringApplication.run(DemoApplication.class, args);
    }
}
```

✕ WHAT CONFUSION DID I HAVE?

Six lines of code felt too small to be responsible for an entire running web application with dozens of beans and a live server. I treated it as an unexplainable "magic switch."

✓ WHAT IS THE CORRECT UNDERSTANDING?

Nothing about this line is magic — it's a regular static method that performs a well-defined, six-stage sequence: create the ApplicationContext, scan for components, apply auto-configuration, register and inject beans, start the embedded server, and finally signal that the application is ready. Every stage is ordinary Java code you could, in principle, read yourself.

◆ WHY DOES SPRING WORK THIS WAY?

Plain Spring (before Spring Boot) required you to configure most of this by hand — an XML file listing beans, a manually deployed servlet container, explicit configuration classes. `SpringApplication.run()` exists specifically to collapse all of that ceremony into a single, sensible-defaults method call.

◆ DID YOU KNOW?

`@SpringBootApplication` is a "meta-annotation" — it's really three annotations bundled into one: `@Configuration` (this class can define beans), `@EnableAutoConfiguration` (turn on Spring Boot's auto-configuration), and `@ComponentScan` (scan this package tree for beans).

STAGE 1

Creating the ApplicationContext

The very first thing `SpringApplication.run()` does is construct an empty `ApplicationContext` — the bean registry from Part II, before it holds a single bean.

✓ WHAT IS THE CORRECT UNDERSTANDING?

Spring Boot inspects your classpath to decide which flavor of context to create — a `AnnotationConfigServletWebServerApplicationContext` for a traditional web app, or a reactive equivalent if WebFlux is present. This decision happens before any of your own beans exist.

Classpath Contains	Context Type Created
spring-boot-starter-web	Servlet-based web context (what this guide covers)
spring-boot-starter-webflux	Reactive web context
Neither	Plain, non-web application context

◆ WHY DOES SPRING WORK THIS WAY?

This is "convention over configuration" in action — instead of you writing code to declare "this is a web app," Spring Boot infers it from what's already sitting on your classpath (which dependencies you added in your build file). One less decision you have to make explicitly.

◆ REAL-WORLD ANALOGY

It's like moving into a new office: before hiring a single employee (bean), the company first decides what kind of building it needs — a retail storefront or a warehouse — based on what kind of business it's actually running.

⚙ INTERNAL PROCESS

Internally, `SpringApplication` keeps a list of "ApplicationContextFactory" implementations, each one checking the classpath for tell-tale classes (like a servlet API class) to decide whether it's the right fit for this particular application.

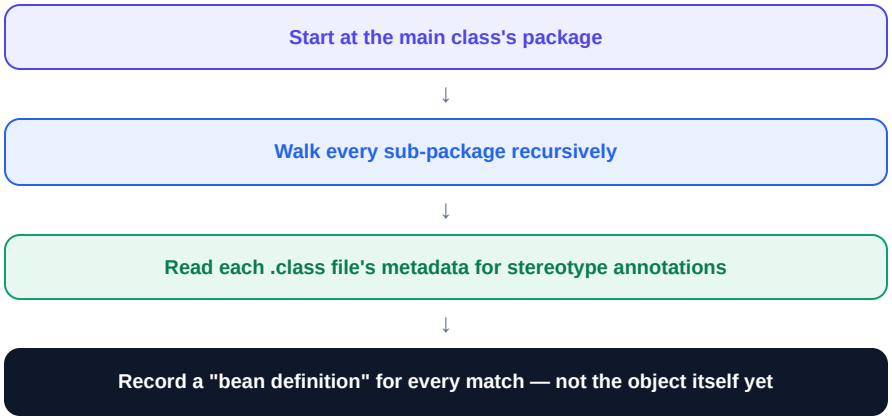
STAGE 2

Classpath Scanning

With an empty context created, Spring now performs the scan described in Part II — but it's worth seeing precisely what "scanning" involves at a mechanical level.

✓ WHAT IS THE CORRECT UNDERSTANDING?

Scanning reads your compiled `.class` files directly — not your source code — using a library called ASM to inspect bytecode metadata without fully loading each class into the JVM. This is done for speed: fully loading every class up front would be slow and wasteful.



◆ DID YOU KNOW?

Scanning produces **bean definitions**, not beans. A bean definition is metadata — the class, its scope, its dependencies — describing how to build the object later. Actual object construction happens in a later stage, not during scanning itself.

⚠ COMMON BEGINNER MISTAKE

Assuming a larger codebase means a proportionally slower scan. In practice, bytecode-level scanning (without loading every class) is fast enough that scan time rarely dominates startup time — auto-configuration and bean initialization usually take longer.

🎓 INTERVIEW TIP

"What's the difference between a bean definition and a bean?" is a strong follow-up question interviewers ask. Answer: a bean definition is metadata describing how to create an object; a bean is the actual, constructed object living in the context.

STAGE 3

Auto Configuration & @EnableAutoConfiguration

This is where the phrase "it just works" comes from. Spring Boot doesn't just scan *your* classes — it also configures dozens of framework-level beans for you, automatically, based on what's on your classpath.

✕ WHAT CONFUSION DID I HAVE?

I never wrote a single line of code registering the embedded Tomcat, the JSON message converter, or the default error page — yet they all worked. It felt like they simply "came with Spring Boot," without a clear mechanism behind that.

✓ WHAT IS THE CORRECT UNDERSTANDING?

`@EnableAutoConfiguration` loads a long list of pre-written `@Configuration` classes bundled inside Spring Boot's own JARs — things like `TomcatAutoConfiguration` and `JacksonAutoConfiguration`. Each one is wrapped in conditions, and only activates if certain classes are present on your classpath and certain beans don't already exist.

```
// simplified real example, from Spring Boot's own source
@Configuration
@ConditionalOnClass(Servlet.class)  // only if servlet API is on the classpath
@ConditionalOnMissingBean(value = ServletWebServerFactory.class)
public class TomcatAutoConfiguration {
    @Bean
    public TomcatServletWebServerFactory tomcatFactory() {
        return new TomcatServletWebServerFactory();
    }
}
```

◆ WHY DOES SPRING WORK THIS WAY?

The `@ConditionalOnMissingBean` check is what makes auto-configuration overridable, not rigid: if you define your own `TomcatServletWebServerFactory` bean, Spring Boot's own default silently steps aside instead of conflicting with yours.

🎓 INTERVIEW TIP

"How would you override an auto-configured bean?" — define your own bean of the same type in a `@Configuration` class. Auto-configuration's `@ConditionalOnMissingBean` guards mean your explicit bean always wins.

STAGE 4

Bean Registration & Dependency Injection

By now, the context holds a complete set of bean *definitions* — both yours (from scanning) and Spring Boot's own (from auto-configuration). This stage is where they finally become real, live objects.

✓ WHAT IS THE CORRECT UNDERSTANDING?

Spring works through the bean definitions and instantiates each one, resolving constructor dependencies as it goes — which means beans with no dependencies get created first, and beans that depend on others wait until their dependencies already exist.

ORDER MATTERS

If `JournalService` needs a `JournalEntryRepository`, Spring must fully construct the repository bean first, then pass it into the service's constructor.

CIRCULAR DEPENDENCIES

If Bean A needs Bean B, and Bean B needs Bean A, neither can finish constructing first. Spring detects this and fails startup with a clear error rather than deadlocking.

```
// this WILL fail at startup with a "circular dependency" error:
@Service
public class A { public A(B b) {} }

@Service
public class B { public B(A a) {} }
```

◆ WHY DOES SPRING WORK THIS WAY?

Failing loudly at startup — instead of allowing a half-built object into production — is a deliberate design choice. A circular dependency almost always signals a real design problem, and Spring surfaces it the moment your app tries to boot, not hours later in production.

◆ DID YOU KNOW?

Field injection (rather than constructor injection) can sometimes mask circular dependencies, because Spring can create the object first and inject fields afterward. This is one more reason constructor injection is the recommended default — it surfaces design problems earlier.

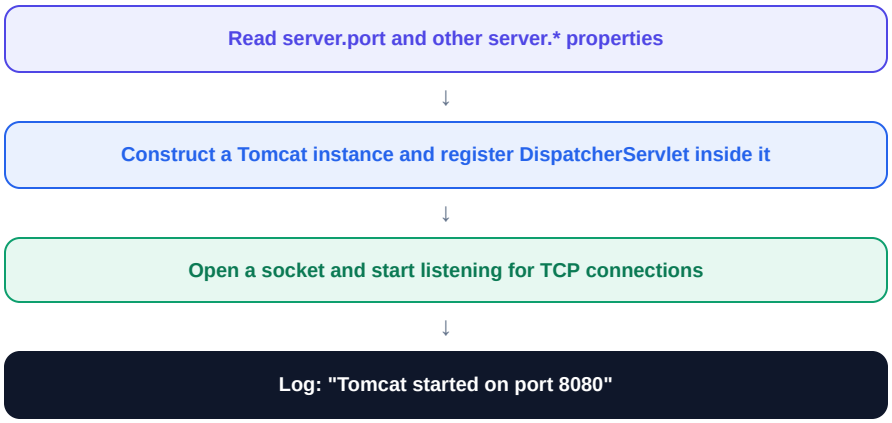
STAGE 5

Embedded Tomcat Startup

All beans now exist in memory — but nothing outside the JVM can talk to your application yet. This stage is what finally opens the door: the embedded Tomcat server (introduced in Part I) actually starts listening on a network port.

✓ WHAT IS THE CORRECT UNDERSTANDING?

One of the beans registered during auto-configuration is a `TomcatServletWebServerFactory`. Now that the context has finished building beans, it calls this factory to actually construct and start a Tomcat instance, bound to `server.port` (default `8080`), with the DispatcherServlet already registered inside it.



◆ WHY DOES SPRING WORK THIS WAY?

Starting the server *after* all beans are built and injected — not before — matters: if a request arrived while beans were still half-constructed, it could hit a Controller whose Service dependency wasn't ready yet. Opening the port last guarantees the whole application is coherent before any traffic can reach it.

⚠ COMMON BEGINNER MISTAKE

Seeing `Port 8080 was already in use` and assuming Spring Boot itself is broken. This error means another process (often a previous run of your own app that didn't shut down cleanly) already has that port open — it's an operating-system-level conflict, not a Spring bug.

◆ DID YOU KNOW?

You can change the port with one line in `application.properties`: `server.port=9090`. Because Tomcat's configuration is itself just a bean built from your properties, no code changes are ever required.

STAGE 6

The Application Ready State

The familiar Spring Boot banner and log lines end with something like `Started DemoApplication in 1.842 seconds`. That message isn't just cosmetic — it marks a real, well-defined milestone in the startup sequence.

✓ WHAT IS THE CORRECT UNDERSTANDING?

Once the context is fully refreshed and Tomcat is listening, Spring Boot publishes two events in sequence: `ApplicationStartedEvent` (context is ready, server is listening) and then `ApplicationReadyEvent` (the application is fully ready to service requests). Your own code can listen for either.

```
@Component
public class StartupListener {

    @EventListener(ApplicationReadyEvent.class)
    public void onReady() {
        System.out.println("Application is fully ready — safe to run startup tasks now.");
    }
}
```

◆ WHY DOES SPRING WORK THIS WAY?

Publishing a distinct "ready" event gives you a safe, guaranteed point to run one-time startup tasks — warming a cache, verifying an external connection, scheduling a job — with full confidence that every bean and the server itself are completely initialized first.

◆ REAL-WORLD ANALOGY

It's the difference between a restaurant finishing its prep work and actually flipping the "OPEN" sign in the window. The kitchen being stocked (beans built) and the doors being unlocked (Tomcat listening) both need to happen — but the sign only flips once both are true.

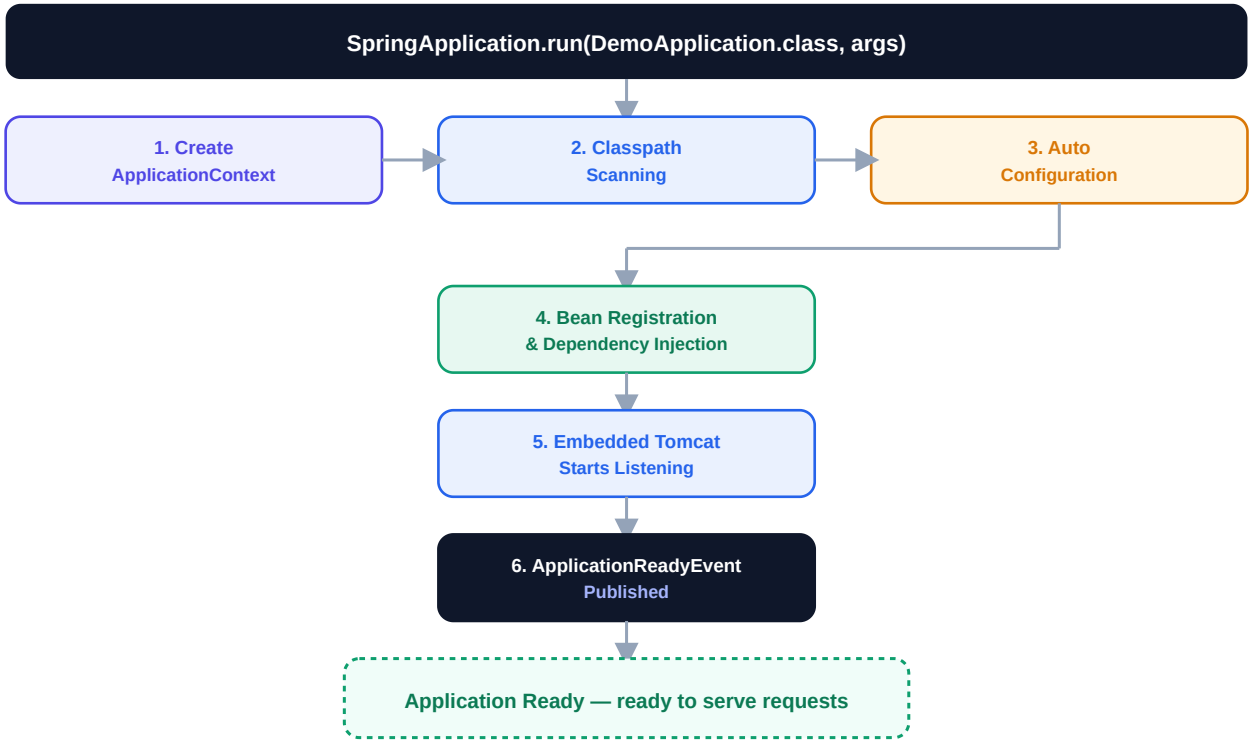
🎓 INTERVIEW TIP

"Where would you put logic that needs to run once, right after the app starts?" — a strong answer names `ApplicationReadyEvent` specifically (over `@PostConstruct`, which runs per-bean, potentially before the whole context and server are ready).

THE FULL PICTURE

The Full Startup Flow, End to End

Six stages, one continuous sequence. This is everything that happens between running `java -jar app.jar` and seeing "Started DemoApplication."



Total elapsed time: typically under 2 seconds for a small Spring Boot app

🎓 **INTERVIEW TIP**

When asked to "explain what happens when a Spring Boot app starts," walk through these exact six stages in order. Most candidates only mention stage 3 (auto-configuration) — naming all six is what demonstrates real depth.

CHAPTER CLOSE

Common Beginner Mistakes & Chapter Summary

⚠ SIX MISTAKES WORTH WATCHING FOR

1. Treating SpringApplication.run() as unexplainable magic. It's a well-defined six-stage sequence — context creation, scanning, auto-configuration, bean wiring, server startup, ready event.

2. Confusing bean definitions with beans. Scanning produces metadata describing how to build an object; actual construction happens later, during bean registration.

3. Assuming auto-configured beans can't be overridden. Most are guarded with @ConditionalOnMissingBean — defining your own bean of the same type simply takes priority.

4. Panicking at "port already in use." This is an operating-system-level conflict from another running process, not a Spring Boot defect.

5. Running startup logic in @PostConstruct when it needs the whole app ready. @PostConstruct runs per-bean, potentially before the server itself is listening — use ApplicationReadyEvent for anything that needs the full application ready.

6. Assuming a larger app always means a much slower scan. Bytecode-level scanning is fast; auto-configuration and bean initialization usually account for more of your startup time.

✎ CHAPTER SUMMARY

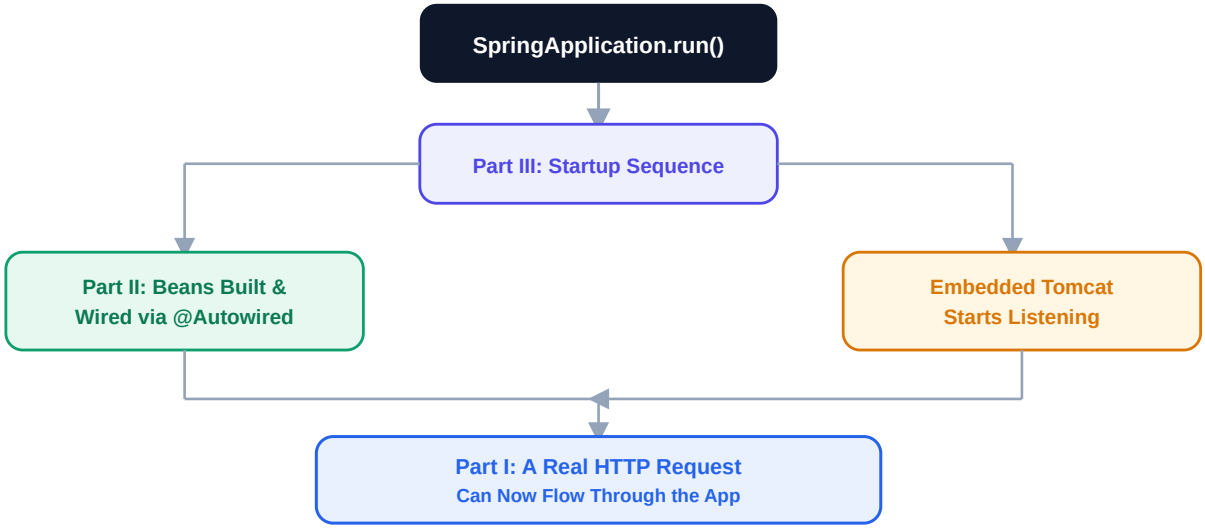
- SpringApplication.run() executes six ordered stages: context creation, classpath scanning, auto-configuration, bean registration & injection, embedded server startup, and the ready event.
- Auto-configuration loads Spring Boot's own pre-written @Configuration classes, activated conditionally based on your classpath and existing beans.
- @ConditionalOnMissingBean is what makes auto-configuration safely overridable by your own beans.
- Beans are constructed in dependency order — beans with no dependencies first, dependents afterward — and circular dependencies fail startup loudly.
- The embedded server only starts listening once all beans are fully built and wired, guaranteeing a coherent application before any request can arrive.
- ApplicationReadyEvent is the safe, guaranteed point to run one-time startup logic.

CLOSING

THE BIG PICTURE

Final Mental Model — How It All Connects

Three concepts, one running application. Here's how every piece from this guide fits together.



◆ THE FULL PICTURE

Part III's startup sequence is what *produces* the wired beans described in Part II. Those wired beans — Controller, Service, Repository — are exactly what Part I's request lifecycle travels through. Nothing in this guide stands alone; each part is the mechanism underneath the next.

✎ KEY TAKEAWAYS FROM THE WHOLE JOURNEY

- Every request travels a fixed twelve-step path — Tomcat, DispatcherServlet, Handler Mapping, Controller, Service, Repository, Database, and back through Jackson.
- @Autowired doesn't create objects — it asks the already-running IoC Container for an existing bean, resolved by type through reflection.
- Beans are singleton by default: one shared instance, injected everywhere it's needed.
- SpringApplication.run() is a well-defined six-stage sequence, not unexplainable magic — and it's what builds the very beans @Autowired hands out.
- Auto-configuration is conditional and overridable — your own beans always take priority over Spring Boot's defaults.
- Every layer and every stage exists to isolate one responsibility — which is exactly why understanding each one in isolation makes the whole framework make sense.

END OF JOURNEY

Three Concepts You'll Never Look At the Same Way Again

The request lifecycle, the secret behind @Autowired, and what really happens at startup are no longer three separate mysteries — they're one connected system, and you now know exactly how it fits together.

◆ WHAT TO EXPLORE NEXT

Now that Generics (the companion guide in this series) and Spring Boot's core mechanics are both demystified, the natural next steps are: Spring AOP and proxies, transaction management with @Transactional, and Spring Security's filter chain — each one builds on the exact same "container does the work for you" pattern covered here.

Written by Muhammad Sufyan

If this guide helped you, let's connect and keep learning in public together.

Connect on LinkedIn →